Carnegie Mellon University

Multi-Agent Adaptive Retriever-Augmented Answer Generation Pipeline for Question and Answering Backed by Reasonings (MAA-RAG)

Abstract

In many domain-specific question-answering system (QA), Retriever-Augmented generation (RAG) has become essential for enhancing answer quality and reducing hallucinations. However, there are some problems associated with a RAG system. The first problem is, RAG depends heavily on the availability and quality of documents in the external knowledge base, as well as the effectiveness of the pipeline. The second problem is, RAG is not always suitable for all questions. For some questions, Graph RAG might be the correct approach, and for some questions, an LLM plus context might be the correct approach. The third problem is, some applications like medical Q/A requires a very high-quality answers backed by a sequence of reasonings, and most pure RAG systems rely heavily on generated snippets and prompting to generate answers. Lastly, most RAG systems use a fixed pipeline, but in the real-world scenario, the nature and the structure of questions are very different, and a fixed pipeline will not be suitable for all question types. To address these challenges, we proposed a multi-agent QA system with an adaptive pipeline from query reformulation to answer-generation. In this system, we defining multiple agents for different tasks such as planning, reasoning, answer-generation, and verification. The planner agent intelligently selects the best suitable pipeline for a specific query, ensuring high-quality answers, avoiding unnecessary approaches for certain queries. During inference, the planner uses Markov Decision Processes (MDP) to decide the workflow, and during training, the planner uses reinforcement learning (RL). The RL in this approach is based on a reward (F1-score) and a cost-based penalty.

General Problems with RAG

- The first problem is, RAG depends heavily on the availability and quality of documents in the external knowledge base, as well as the effectiveness of the pipeline.
- RAG is not always suitable for all questions. For some questions, Graph RAG might be the correct approach, and for some questions, an LLM plus context might be the correct approach.
- Some applications like medical Q/A requires a very high-quality answers backed by a sequence of reasonings and verifications, and most pure RAG systems rely heavily on generated snippets and prompting to generate answers.
- Most RAG systems use a fixed pipeline, but in the real-world scenario, the nature and the structure of questions are very different, and a fixed pipeline will not be suitable for all question types.

Motivations

- **Reinforcement Learning in RAG Systems** [1][2][3][4] – Using RL guided by reward and penalty to learn learn optimal policy that an agent can use in MDP to take the best actions.
- **Semi-Markov Decision Processes (SMDP)** [2][5] – Useful for systems where decisions are made at random points in time rather than at fixed intervals, allowing for actions to take variable amounts of time to complete. Q/A systems are an example.

- **Hierarchical Leaning Instead of Flat PPO** – Hierarchical learning reduces complexity and high variance in large-state space. Instead of computing single policy for all tasks, HL decomposes complex tasks into smaller, manageable subtasks or levels, enhancing computational efficiency and enabling agents to tackle long-horizon problems with sparse rewards. [2][4]

- Reasoning Adds Layer of Answer Quality

Related Work

- **Baseline Paper (MAO-RAG)** [3]**:** Proposes an adaptive RAG framework called MAO-ARAG, which leverages multi-agent orchestration. This approach specifically define multiple executor agents, representing typical RAG modules such as query reformulation agents, document selection agent, and generation agents. A planner agent intelligently selects and integrates the appropriate agents from these executors into a suitable workflow tailored for each query, striving for high-quality answers while maintaining reasonable costs.

- **DRL-RAG** [1]**:** A Dynamic Reinforcement Learning-Driven RAG Method for Enhancing Language Model Capabilities. DRL-RAG utilizes relevance scores from a cross-encoder as a reward signal within a reinforcement learning (RL) mechanism to optimize the quality of retrieved documents.

- **Retrieval-Augmented Hierarchical in-Context Reinforcement Learning and Hindsight Modular Reflections for Task Planning with LLMs** [4]: Proposes Retrieval-Augmented Hierarchical in-context reinforcement Learning (RAHL), a novel framework that decomposes complex tasks into sub-tasks using an LLM-based high-level policy, in which a complex task is decomposed into sub-tasks by a high-level policy on-the-fly.

Gaps from SOTAs and Baseline in Related Works

- Flat PPO
- Design complexity due to too many agents. In MAO-RAG [3], there are 7 executor agents, and 1 orchestrator that select actions by triggering the agents.
    - **Problem**: Latency and complexity. Why have separate agents for query decomposition, query rewriter, query rewriter, document selector, etc., when all these could be done by a single agent?
- Adaptation is only found at query reformulation and document selection stages. What about answer generation stage?
- Fixed answer-generation technique (mostly regular RAG). Why not add different agents for different types of query? Ex. Graph RAG, RAG, context + pure LLM
- The RL-based SOTA systems do not consider reasoning and answer verification.

Hypothesis

- A RAG with adaptive pipeline and a reasoning block infused in the pipeline will perform better than a RAG with a fixed pipeline, a RAG with reasoning infused but fixed pipeline, and a RAG with an adaptive pipeline but no reasoning infused.

- A RAG with few agents that are capable to take actions from a large action space will have better latency and design costs than a RAG that has multiple agents (each with smaller action space).

Proposed Method

Planner Agent

- Planner Agent – The planner agent in this proposed method is an adopted planner agent from MAO-RAG[3].
- Unlike in the MAO-RAG[3] whose action is to select an agent that perform a specific query reformulation[3], the planner agent in this approach selects and executes the action in its action space, reducing latency and design complexity.
- The agent is the orchestrator, and it uses Hierarchical Semi-Markov Decision Processes (HSMDP) to design the workflow based on the query.
- We adopted the following actions from MAO-RAG[3]: query decomposition serial (QDS), query decomposition parallel (QDP), query writer (QR), document selector (DS), retriever agent (RA), answer generating agents (RAG, Graph RAG, and context-clue LLM), and answer summarization (AS). But instead of making QDS, QDP, QR, and QS separate agents, we make this a single agent with ability to perform all these actions from its internal algorithm. Also, instead of separating AG and AS, we also make this a single agent as well. In addition to the agents adopted from MAO-RAG[3], we add a reasoning agent (RA) and a verification agent (VA).
- New state space used in our approach: $\mathrm{A}_{\mathrm{planner}} = \{\mathrm{QDS}, \mathrm{QDP}, \mathrm{QR}, \mathrm{DS}, \mathrm{RA}, \mathrm{AS}, \mathrm{AG}, \mathrm{AS}, \mathrm{RAG}, \mathrm{GRAG}, \mathrm{LLM}\}$

Proposed Method

Reasoning Agent

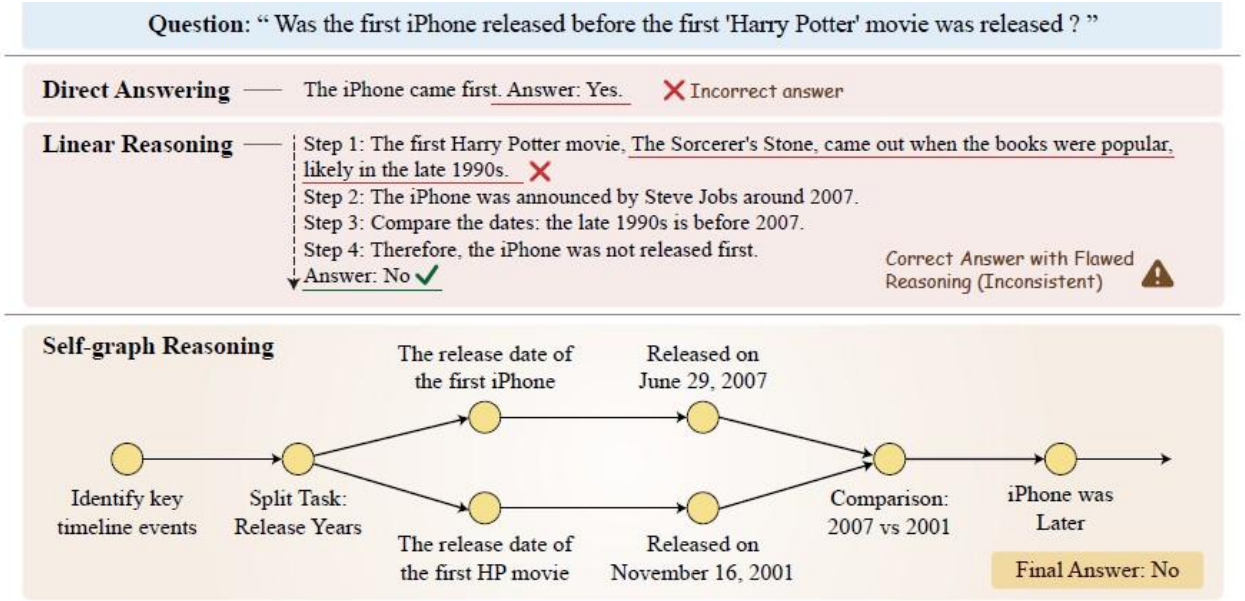
- Before preparing the final prompt for the LLM, the reasoning agent takes in the reformulated query and generate a self-graph reasoning.

- In our approach, we adopt a Self-Graph Reasoning (SGR) from [6], a framework designed to enable LLMs to externalize their latent reasoning process into a structured graph before generating the final answer.

- By explicitly organizing intermediate reasoning into a graph structure, SGR provides clearer logical constraints and reduces the drift and inconsistency often seen in linear Chain-of-Thought reasoning.

Proposed Method

Reasoning Agent



An illustrative example of the Self-Graph Reasoning framework adopted from [6]

Answer-Generating Agents

- In this approach, we use three popular answer-generating agents based on different answer-generating approach.
- RAG – this agent relies on a large corpus of documents to extract snippets to guide an LLM response.
- Graph RAG – This agent utilizes a structured knowledge graphs instead of relying solely on vector search.
- Context-clue LLM – this agent relies on user-provided context plus the query to answer the question.
- While these are separate agents, to the planner agent, selecting any of them is an action in the action space.

Verification Agent

- Checks for correctness, logical consistency, and hallucinations.
- It improves trustworthiness in answering questions where highly accuracy and reliable is needed. Ex. Medical QA, diagnosis, legal QA, scientific QA, etc.
- This agent uses NLI to recheck facts in generated answer against retrieved documents. This is needed mostly for factoid questions.
- For a claim generated from the answer and decomposed into $C_i = \{c_1 + c_2 + \cdots + c_n\}$ and a retrieved supporting evidence, $D_j = \{d_1 + d_2 + \cdots + d_n\}$, an answer is verified with

$$\mathrm{NLI}(\mathrm{d_j}, \mathrm{c_i})$$

Mathematical Formulation

Problem Definition

We formulate adaptive Retrieval-Augmented Generation (RAG) as a sequential decision-making problem in which an intelligent agent dynamically determines which operations to perform, in what order, and when to terminate, in order to maximize answer quality while minimizing computational cost and latency. This problem is naturally modeled as a *Hierarchical Semi-Markov Decision Process (HSMDP)*.

SMDP Formulation

The environment is defined as a semi-Markov decision process:

$$\mathcal{M} = (\mathcal{S}, \mathcal{Z}, \mathcal{P}, \mathcal{R}, \gamma), \tag{1}$$

where:

- $\mathcal{S}$ denotes the state space,
- $\mathcal{Z}$ denotes the set of high-level actions (options),
- $\mathcal{P}$ denotes the transition probability function,
- $\mathcal{R}$ denotes the reward function,
- $\gamma \in (0, 1]$ is the discount factor.

State Space

At time t, the agent observes the state:

$$s_t = (q, H_t, D_t, A_t, C_t), \tag{2}$$

where:

- q is the user query,
- H_t is the interaction history,

- D_t is the retrieved document set,
- A_t is the partial generated answer,
- C_t represents the current verification confidence.

Hierarchical Action Space

We define a hierarchical action space:

$$\mathcal{Z} = \{z_1, z_2, \ldots, z_K\}, \tag{3}$$

where each $z \in \mathcal{Z}$ corresponds to a macro-action:

$$\mathcal{Z} = \{\text{retrieve}, \text{reason}, \text{verify}, \text{rewrite}, \text{expand}, \text{terminate}\}. \tag{4}$$

Each macro-action activates a low-level policy $\pi_L(a_t \mid s_t, z_t)$, where $a_t \in \mathcal{A}$ represents primitive actions such as query formulation, document ranking, reasoning steps, and verification operations.

Hierarchical Policy Structure

The hierarchical agent consists of two policies:

High-level policy (meta-controller):

$$\pi_H(z_t \mid s_t), \tag{5}$$

Low-level policy (controller):

$$\pi_L(a_t \mid s_t, z_t). \tag{6}$$

The joint policy is:

$$\pi(a_t, z_t \mid s_t) = \pi_H(z_t \mid s_t) \cdot \pi_L(a_t \mid s_t, z_t). \tag{7}$$

Options Framework

Each macro-action z is formalized as an option:

$$z = (\mathcal{I}_z, \pi_L^z, \beta_z), \tag{8}$$

where:

- $\mathcal{I}_z$ is the initiation set,
- π_L^z is the intra-option policy,
- $\beta_z(s)$ is the termination probability.

Transition Dynamics

Let τ denote the duration of executing option z. The SMDP transition is:

$$\mathcal{P}(s', \tau \mid s, z) = \sum_{a_{t:t+\tau}} P(s', \tau \mid s, a_{t:t+\tau}), \tag{9}$$

where:

$$a_{t:t+\tau} \sim \pi_L(\cdot \mid s_t, z). \tag{10}$$

Reward Function

We define the reward at time t as:

$$r_t = r_{\text{ans}} - \lambda_c C_t - \lambda_l L_t - \lambda_h H_t, \tag{11}$$

where:

- r_{ans} measures answer quality,
- C_t denotes computational cost,
- L_t denotes latency,
- H_t denotes hallucination penalty,

- $\lambda_c, \lambda_l, \lambda_h$ are weighting coefficients.

Answer Quality via NLI-Based Verification

The answer is decomposed into N atomic claims $\{c_i\}_{i=1}^N$, each verified against retrieved evidence $\{e_j\}_{j=1}^M$ using Natural Language Inference (NLI):

$$v_{ij} = \text{NLI}(e_j, c_i), \tag{12}$$

where:

$$v_{ij} \in \{+1, 0, -1\}. \tag{13}$$

The verification score is:

$$V = \frac{1}{N} \sum_{i=1}^{N} \max_j v_{ij}. \tag{14}$$

The reward is augmented as:

$$r_t \leftarrow r_t + \lambda_v V. \tag{15}$$

Hierarchical PPO Optimization

The optimization objective is:

$$\max_{\theta_H, \theta_L} \mathbb{E} \left[\sum_{t=0}^{T} \gamma^t r_t \right]. \tag{16}$$

High-level PPO objective:

$$L_H(\theta_H) = \mathbb{E}_t \left[\min \left(\rho_t^H A_t^H, \text{clip}(\rho_t^H, 1-\epsilon, 1+\epsilon) A_t^H \right) \right], \tag{17}$$

Low-level PPO objective:

$$L_L(\theta_L) = \mathbb{E}_t \left[\min \left(\rho_t^L A_t^L, \text{clip}(\rho_t^L, 1-\epsilon, 1+\epsilon) A_t^L \right) \right]. \tag{18}$$

Value Functions

$$V_H(s_t) = \mathbb{E}\left[\sum_{k=0}^{K} \gamma^k r_{t+k}\right], \tag{19}$$

$$V_L(s_t, z_t) = \mathbb{E}\left[\sum_{k=0}^{\tau} \gamma^k r_{t+k}\right]. \tag{20}$$

Overall Algorithm

Algorithm 1 Hierarchical SMDP-Based Adaptive RAG

```
Initialize π_H, π_L, and value networks each episode
Observe query q, initialize s_0 not terminated
Sample z_t ~ π_H(z_t | s_t)
Execute z_t using π_L(a_t | s_t, z_t)
Observe s_{t+τ}, r_t
Store transition
Update π_H, π_L using PPO
```

References

[1] N. Zhang, R. Ye, and J. Han, "DRL-RAG: A Dynamic Reinforcement Learning-Driven RAG Method for Enhancing Language Model Capabilities," in 2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA), Shenyang, China: IEEE, Oct. 2025, pp. 1161–1166. doi: 10.1109/ISPA67752.2025.00159.

[2] M. Ghavamzadeh, S. Mahadevan, and R. Makar, "Hierarchical Multi-Agent Reinforcement Learning".

[3] Y. Chen et al., "MAO-ARAG: Multi-Agent Orchestration for Adaptive Retrieval-Augmented Generation," Aug. 01, 2025, arXiv: arXiv:2508.01005. doi: 10.48550/arXiv.2508.01005.

[4] C. Sun, S. Huang, H. Liu, J. Gong, and D. Pompili, "Retrieval-Augmented Hierarchical in-Context Reinforcement Learning and Hindsight Modular Reflections for Task Planning with LLMs," in 2025 IEEE International Conference on Robotics and Automation (ICRA), Atlanta, GA, USA: IEEE, May 2025, pp. 1217–1224. doi: 10.1109/ICRA55743.2025.11128105.

[5] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," Artificial Intelligence, vol. 112, no. 1–2, pp. 181–211, Aug. 1999, doi: 10.1016/S0004-3702(99)00052-1.

[6] Y. Chen et al., "From Chains to Graphs: Self-Structured Reasoning for General-Domain LLMs," Jan. 20, 2026, arXiv: arXiv:2601.03597. doi: 10.48550/arXiv.2601.03597.